

BigQuery ML

O modelo já existe. Ele presta?

O `CREATE MODEL` é uma linha de SQL. A qualidade do modelo é decidida nas etapas que vêm antes dele.

Aula de **04/09/2026** · Prof. **Guilherme** · Dataset: **tb_anuncios_silver** (1.000 anúncios, Goiânia) · Guia de estudos e referência para o Trabalho 1

0 Ponto de partida — o que já temos

A aula de 28/08 terminou no primeiro `CREATE MODEL`. Este material parte exatamente dali: revisar o que foi construído e conferir os resultados.

Nas aulas do Sávio foi construído o caminho completo: **raw** → **bronze** → **silver** nos notebooks, depois a `gold_predicao_preco`, e por fim o `CREATE MODEL` do `modelo_preco_imoveis`. O treino terminou sem erro — mas nenhuma métrica foi verificada ainda:

O `modelo_preco_imoveis` que está no seu projeto... **é bom?**

O material começa respondendo isso, em uma query.

Todo o conteúdo se apoia no mesmo esqueleto de três comandos:

o esqueleto

```
CREATE MODEL ...    -- treina
ML.EVALUATE(...)    -- vale alguma coisa?
ML.PREDICT(...)     -- usa
```

Se em algum momento o detalhe confundir, volte a este esqueleto. É sempre esse desenho.

O que você precisa ter aberto

- BigQuery Studio, no **seu** projeto GCP (o mesmo das aulas do Sávio)
- O dataset `anuncios` com a `tb_anuncios_silver` criada nos notebooks
- Os scripts da aula: `00` a `07`, na pasta `sql/` do repositório

Quer ver o destino antes de sair? O caminho inteiro de hoje — do bucket até os quatro modelos comparados — está desenhado em `pipeline.html`. Vale abrir agora para se localizar, e vale voltar nele mais tarde, quando cada caixa já tiver sido construída por você.

Os códigos usam SEU_PROJETO. Digite o ID do seu projeto GCP no campo da barra do topo e clique em **OK**: todos os códigos desta página passam a mostrar o seu ID, e o botão **copiar** já copia a query

pronta. Nos arquivos `.sql` do repositório o ajuste continua manual. Sem a silver no projeto? Plano B: peça o `tb_anuncios_silver.csv` e faça upload direto no BigQuery.

Sobre os valores de referência. Todos os números deste material foram medidos sobre o dataset oficial da disciplina — o mesmo `anuncios.csv` distribuído na Turing, processado pelos notebooks do Sávio. Se a sua silver veio desse mesmo caminho, os resultados devem bater. Se algum número divergir, vale investigar antes de seguir: pode ter havido diferença na carga ou na montagem do dataset. A única variação esperada é no modelo de árvore do bloco 10, que muda um pouco a cada treino.

Um modelo não avisa quando é ruim. A avaliação é uma etapa explícita — e vem antes de qualquer uso.

Script: `00_avaliar_modelo_de_hoje.sql`. Uma query, sobre o modelo que **você** treinou:

a linha de verdade

```
SELECT * FROM ML.EVALUATE(MODEL `SEU_PROJETO.anuncios.modelo_preco_imoveis`);
```

As duas colunas que importam hoje:

Métrica	O que é
<code>mean_absolute_error</code>	Em reais : quanto o palpite erra, em média
<code>r2_score</code>	Quanto da variação do preço o modelo explica. 1 = perfeito · 0 = empata com chutar a média · negativo = pior que chutar a média

► **Resultado esperado** — não abra antes de rodar

A pergunta que estrutura o material inteiro:

De quem é a culpa — **do algoritmo ou do dado**? O que você faria para melhorar? Anote suas ideias antes de seguir: os próximos blocos percorrem exatamente essas frentes.

Faltou em 28/08? O PASSO 0 do script traz o CREATE MODEL exato do Sávio, comentado — descomente e rode (~1 min) antes do ML.EVALUATE.

Checkpoint 1. Meu ML.EVALUATE do modelo_preco_imoveis retornou e eu anotei o MAE e o R² do meu modelo.

Pela arquitetura medalhão, limpar é responsabilidade da silver. Antes de retreinar, vale conferir se a nossa cumpriu esse papel.

Na arquitetura medalhão de referência, a limpeza — nulos, outliers, duplicatas — acontece na passagem bronze → silver. A silver construída nos notebooks chegou estruturada (23 colunas tipadas, zero JSON), mas a etapa de limpeza ficou em aberto — pode ter sido decisão do pipeline, pode ter sido um detalhe que passou na montagem do dataset. Script: `01_silver_mente.sql`. O modelo é pior que chutar a média — três verificações mostram o que a tabela esconde.

Verificação 1 — o preço

estatística descritiva

```
SELECT
  COUNT(*)                AS anuncios,
  ROUND(MIN(preco))        AS menor_preco,
  ROUND(APPROX_QUANTILES(preco, 100) [OFFSET(50)]) AS mediana,
  ROUND(AVG(preco))        AS media,
  ROUND(MAX(preco))        AS maior_preco,
  ROUND(STDDEV(preco))     AS desvio_padrao
FROM `SEU_PROJETO0.anuncios.tb_anuncios_silver`
WHERE preco IS NOT NULL;
```

► **Resultado esperado** — não abra antes de rodar

Para dimensionar o impacto, meça — não estime:

o número que importa

```
SELECT
  COUNTIF(preco > 50000000) AS anuncios_suspeitos,
  COUNT(*)                  AS total,
  ROUND(STDDEV(preco))      AS desvio_com_eles,
  ROUND(STDDEV(IF(preco <= 50000000, preco, NULL))) AS desvio_sem_eles
FROM `SEU_PROJETO0.anuncios.tb_anuncios_silver`
WHERE preco IS NOT NULL;
```

29 linhas (2,9%) controlam 90% da variação. Desvio com elas: R\$ 72 mi. Sem elas: R\$ 6,9 mi.

O truque que vale para a vida: o IF por dentro do agregado mede o "com" e o "sem" na mesma passada, sem apagar nada. WHERE decide quais **linhas** entram na conta; IF dentro do agregado decide quais **valores**.

Verificação 2 — a área

áreas impossíveis

```
SELECT bairro, titulo, area_util, area_total, ROUND(preco) AS preco
FROM `SEU_PROJETO.anuncios.tb_anuncios_silver`
WHERE area_util > 100000
ORDER BY area_util DESC
LIMIT 10;
```

► **O campeão** — não abra antes de rodar

A lição. Nenhum modelo vai te avisar que o número é impossível. 459800000 é um número perfeitamente válido para o BigQuery. Quem sabe que uma fazenda não ocupa 63% de Goiânia é **você**, não ele.

Verificação 3 — colunas que parecem features e não são

as colunas mortas

```
SELECT
  COUNT(DISTINCT tipo_contrato) AS tipos_contrato,
  COUNT(DISTINCT status) AS status_distintos,
  COUNT(DISTINCT cidade) AS cidades
FROM `SEU_PROJETO.anuncios.tb_anuncios_silver`;
```

► **Resultado esperado** — não abra antes de rodar

Três queries, três problemas, na tabela que o pipeline inteiro aprovou.

Tipar a coluna não conserta o valor que está dentro dela.

Checkpoint 2. As três queries retornaram no meu projeto e eu sei dizer os três problemas de cabeça: preço, área, colunas mortas/coordenadas.

► **Desafios**

Gold é processo, não prefixo de nome de tabela.

Script: `02_gold_etl.sql`. A `gold_predicao_preco` de 28/08 só removeu NULLs — o imóvel de R\$ 1,4 bi e a fazenda passaram direto para o treino. Nesta seção é construída a tabela em que dá para confiar. Três tipos de decisão:

Decisão	Quando	Exemplo aqui
CORTAR	erro irreversível — não dá para saber o valor certo	preço de R\$ 1,4 bi (era 1,4 mi? 140 mil?)
CONSERTAR	erro reversível — o valor certo é conhecido	lat/lon trocadas (o outro campo tem o valor)
DERIVAR	coluna nova nasce de coluna velha	<code>eh_comercial</code> , <code>preco_por_m2</code>

O momento de NLP: a coluna que a silver perdeu

O raw tinha `uso` (RESIDENTIAL/COMMERCIAL). A silver não tem mais. Sobrou **uma** coluna de texto — e o título sabe:

REGEXP_CONTAINS — isso é NLP

```
SELECT
  COUNTIF(REGEXP_CONTAINS(LOWER(titulo),
    r'comercial|galp[aão]|pr[eé]dio|loja|escrit[oó]rio|barrac[aão]|industrial'))
  AS comerciais,
  COUNT(*) AS total
FROM `SEU_PROJETO.anuncios.tb_anuncios_silver`;
-- Esperado: 179 de 1.000 = 17,9%
```

Não precisa de GPU para começar NLP. Precisa de uma pergunta e de uma expressão regular. `galp[aão]` casa "galpao" e "galpão" — dado real tem gente que digita sem acento. Embedding e LLM são a versão sofisticada disso; ficam no cardápio.

A Gold — cada linha do WHERE é uma decisão assinada

02_gold_etl.sql — o CREATE TABLE

```
CREATE OR REPLACE TABLE `SEU_PROJETO0.anuncios.anuncios_gold` AS
SELECT
  id_anuncio, preco, area_util, quartos, banheiros, suites,
  garagem, condominio, iptu, bairro,

  -- CONSERTAR: devolve cada coordenada ao seu lugar
  CASE WHEN latitude < -40 THEN longitude ELSE latitude END AS latitude,
  CASE WHEN longitude < -40 THEN longitude ELSE latitude END AS longitude,

  -- DERIVAR: o alvo de classificação que a silver não tem mais
  REGEXP_CONTAINS(LOWER(titulo),
    r'comercial|galp[aão]|pr[eé]dio|loja|escrit[oó]rio|barrac[aão]|industrial')
    AS eh_comercial,

  ROUND(preco / area_util, 2) AS preco_por_m2,

  titulo    -- carregada de propósito. Vai dar problema. Spoiler: bloco 7.

FROM `SEU_PROJETO0.anuncios.tb_anuncios_silver`
WHERE
  preco IS NOT NULL                -- sem alvo não há treino
  AND preco BETWEEN 50000 AND 50000000 -- corta erro de escala (29 + 6 linhas)
  AND area_util BETWEEN 20 AND 10000  -- corta unidade trocada (83 + 1 linhas)
;
```

Conferência — **não é opcional**. Um filtro errado não dá erro, dá silêncio:

conferir a Gold

```
SELECT
  COUNT(*)                AS anuncios,
  ROUND(APPROX_QUANTILES(preco, 100)[OFFSET(50)]) AS mediana,
  ROUND(AVG(preco))        AS media,
  COUNTIF(eh_comercial)    AS comerciais
FROM `SEU_PROJETO0.anuncios.anuncios_gold`;
```

► **Resultado esperado** — não abra antes de rodar

Para discutir:

Os cortes de preço e área são regras de negócio disfarçadas de SQL. Quem, numa empresa de verdade, deveria assinar embaixo do "50 milhões" — você ou a área de negócio?

Checkpoint 3. Minha `anuncios_gold` existe e o `COUNT(*)` deu **887**. Se deu outro número, meu filtro está diferente — resolver ANTES de treinar, ou os meus números não vão bater com os do quadro.

▶ Desafios

4 O cardápio do BigQuery ML

Primeiro a pergunta. Depois o `model_type`. Nunca o contrário.

Sua pergunta	Nome disso	<code>model_type</code>
Quanto vai ser? (um número)	regressão	<code>LINEAR_REG</code> , <code>BOOSTED_TREE_REGRESSOR</code>
É A ou B?	classificação	<code>LOGISTIC_REG</code> , <code>BOOSTED_TREE_CLASSIFIER</code>
Quanto no mês que vem?	série temporal	<code>ARIMA_PLUS</code>
Quais se parecem entre si?	agrupamento	<code>KMEANS</code>
O que recomendar?	recomendação	<code>MATRIX_FACTORIZATION</code>
Buscar por significado	embeddings	<code>ML.GENERATE_EMBEDDING</code> + <code>VECTOR_SEARCH</code>
Chamar o Gemini de dentro do SQL	LLM	<code>ML.GENERATE_TEXT</code> (via Vertex AI)
Já tenho um modelo pronto	importar	<code>TENSORFLOW</code> , <code>ONNX</code> , <code>REMOTE</code>

Por que `ARIMA_PLUS` não é demonstrado hoje. `data_insercao` é a data de *publicação do anúncio*, não a data de venda. Não existe série temporal aqui. **O dado decide o modelo** — não o contrário. Querer ARIMA num dado sem série é o erro que mais aparece em trabalho de disciplina.

O que **não** cabe no BigQuery ML: visão computacional, arquitetura customizada, treino distribuído em GPU. Isso é Vertex AI, e é a aula de 02/10.

Nesta aula são usadas **quatro** linhas dessa tabela: `LINEAR_REG` (bloco 5), `LOGISTIC_REG` (bloco 6), `BOOSTED_TREE_REGRESSOR` e `KMEANS` (bloco 10). Trocar de família é trocar **uma string**.

Nenhuma linha do CREATE MODEL mudou de ideia. O que mudou foi o dado.

Script: `03_regressao.sql`. A mesma pergunta de 28/08 — *quanto vale este imóvel?* — agora sobre a Gold:

03_regressao.sql — treinar

```
CREATE OR REPLACE MODEL `SEU_PROJETO0.anuncios.modelo_preco`  
OPTIONS (  
  model_type           = 'LINEAR_REG',  
  input_label_cols     = ['preco'],    -- <<< o que queremos prever  
  data_split_method    = 'AUTO_SPLIT',  
  enable_global_explain = TRUE         -- só liga ANTES do treino  
) AS  
SELECT  
  preco,                -- label  
  area_util, quartos, banheiros, suites, garagem,  
  condominio, iptu, bairro, eh_comercial  
FROM `SEU_PROJETO0.anuncios.anuncios_gold`;
```

Três coisas para reparar enquanto o treino roda (~1–2 min):

1. **Não existe lista de features.** `input_label_cols` diz qual coluna é a resposta; todo o resto do `SELECT` vira entrada. Engenharia de features = editar o `SELECT`.
2. **bairro é texto e entrou direto.** Em Python: `OneHotEncoder` + `ColumnTransformer`. Aqui, você só colocou a coluna.
3. **Olhe o que ficou de fora:** `id_anuncio` (identificador não é atributo), `preco_por_m2` (contém o próprio preço!), lat/long (37,7% NULL — o BQ imputaria um "centro da cidade" falso) e `titulo` (por quê? anote seu palpite — blocos 7 e 8).

avaliar

```
SELECT * FROM ML.EVALUATE(MODEL `SEU_PROJETO0.anuncios.modelo_preco`);
```

► **Resultado esperado — v0 × v1** — não abra antes de rodar

prever — ordenado pelos PIORES erros, de propósito

```

SELECT
  bairro, eh_comercial, area_util, quartos,
  ROUND(preco) AS preco_real,
  ROUND(predicted_preco) AS preco_previsto,
  ROUND(predicted_preco - preco) AS erro
FROM ML.PREDICT(
  MODEL `SEU_PROJETO0.anuncios.modelo_preco`,
  (SELECT * FROM `SEU_PROJETO0.anuncios.anuncios_gold`)
)
ORDER BY ABS(predicted_preco - preco) DESC
LIMIT 20;

```

Olhe os 20 piores erros. O que eles têm em comum?

No dado real: mansões do Aldeia do Vale, bairros com poucos anúncios — e **anúncios duplicados** (o mesmo imóvel 4x). Isso é análise de erro, e vale mais que o R^2 . (A duplicata volta no cardápio de ideias do Trabalho 1.)

perguntar ao modelo o que pesou

```

SELECT * FROM ML.GLOBAL_EXPLAIN(MODEL `SEU_PROJETO0.anuncios.modelo_preco`)
ORDER BY attribution DESC;

```

► **Resultado esperado** — não abra antes de rodar

O modelo não é uma caixa preta por natureza.
Ele é uma caixa preta quando você não pergunta.

Prever um imóvel que não existe

o modelo virou uma função consultável

```

SELECT ROUND(predicted_preco) AS preco_estimado
FROM ML.PREDICT(
  MODEL `SEU_PROJETO0.anuncios.modelo_preco`,
  (SELECT
    180 AS area_util, -- INT64, como na silver (180.0 dá erro de coerção)

```

```
3      AS quartos,  
2      AS banheiros,  
1      AS suites,  
2      AS garagem,  
800.0  AS condominio,  
2400.0 AS iptu,  
'Setor Bueno' AS bairro,  
FALSE      AS eh_comercial  
)  
);
```

► **Resultado esperado** — não abra antes de rodar

Checkpoint 4. O `modelo_preco` aparece no painel do meu dataset, o `ML.EVALUATE` bateu com o quadro e eu anotei a comparação `v0 → v1`.

► **Desafios**

6 Classificação: comercial ou residencial?

MÃO NA MASSA

Acurácia só faz sentido comparada com a classe majoritária.

Script: `04_classificacao.sql`. O alvo é o `eh_comercial` que **você** criou no bloco 3. Abra o script anterior do lado: mudou **uma** palavra no `model_type` e uma no `input_label_cols`. Todo o resto é igual.

Antes de treinar: a linha de corte

Se eu criar um "modelo" que responde RESIDENCIAL para tudo, sem olhar nada, qual a acurácia dele?

Resposta na Gold: **83,5%** (146 comerciais em 887 = 16,5%). Anote. É a nota mínima — um modelo com 84% num problema onde chutar dá 83,5% aprendeu quase nada.

04_classificacao.sql — treinar

```
CREATE OR REPLACE MODEL `SEU_PROJETO.anuncios.modelo_comercial`  
OPTIONS (  
  model_type          = 'LOGISTIC_REG',      -- <<< era LINEAR_REG  
  input_label_cols    = ['eh_comercial'],    -- <<< era preco  
  data_split_method   = 'AUTO_SPLIT',  
  enable_global_explain = TRUE  
) AS  
SELECT  
  eh_comercial,      -- label  
  preco,             -- agora o preço pode ser feature!  
  area_util, quartos, banheiros, suites, garagem,  
  condominio, iptu, bairro  
FROM `SEU_PROJETO.anuncios.anuncios_gold`;
```

Label e feature são papéis, não propriedades da coluna. O preço que era resposta no bloco 5 é pergunta neste.

A regra nova — e ela vale nota no Trabalho 1. O alvo nasceu do título (a regex do bloco 3), então o título — e qualquer coluna derivada dele — **não pode ser feature deste modelo**. Senão ele não

aprende o que é um imóvel comercial: ele reaprende a SUA regex, com acurácia perfeita e utilidade zero. *"Se o rótulo veio de uma coluna, essa coluna está proibida de entrar no modelo."*

avaliar contra a régua + matriz de confusão

```
SELECT * FROM ML.EVALUATE(MODEL `SEU_PROJETO.anuncios.modelo_comercial`);

SELECT * FROM ML.CONFUSION_MATRIX(MODEL `SEU_PROJETO.anuncios.modelo_comercial`);
```

► **Resultado esperado** — não abra antes de rodar

Precision e recall respondem perguntas diferentes.
Escolher qual importa não é decisão do modelo. É sua.

Filtro de spam: precision (não jogue e-mail bom fora). Exame de câncer: recall (não deixe doente passar).

os erros cometidos com MAIS confiança

```
SELECT
  bairro, area_util, quartos, ROUND(preco) AS preco,
  eh_comercial AS verdade,
  predicted_eh_comercial AS palpite,
  ROUND((SELECT p.prob FROM UNNEST(predicted_eh_comercial_probs) p
        WHERE p.label = predicted_eh_comercial), 3) AS confianca
FROM ML.PREDICT(
  MODEL `SEU_PROJETO.anuncios.modelo_comercial`,
  (SELECT * FROM `SEU_PROJETO.anuncios.anuncios_gold`)
)
WHERE predicted_eh_comercial != eh_comercial
ORDER BY confianca DESC
LIMIT 15;
```

Errando com 51%, o modelo está em dúvida — problema de dado ambíguo.
Errando com 98%, ele aprendeu a coisa errada — problema de feature.

E olhe as linhas: um "residencial" de 800 m² com 0 quartos e preço de galpão... será que a **regex** errou o rótulo, e o modelo está certo? Também acontece — chama-se **ruído de label**.

▶ **ML.GLOBAL_EXPLAIN** — o que pesou — não abra antes de rodar

▶ **Desafios**

7 NLP que funciona: features escondidas no título

MÃO NA MASSA

A intuição do especialista é uma hipótese, não um fato. Ela entra no pipeline depois do teste.

Script: `05_features_texto.sql`. O modelo de preço parou em R^2 0,30 e ainda erra R\$ 2,58 mi. A pergunta do professor Sávio:

"Será que o texto do anúncio não explica parte do preço? Um lugar requintado, com piscina, mobiliado — isso justifica valor. **Dá pra tirar isso do título?**"

Dá. Mas hipótese razoável não é hipótese confirmada: o primeiro passo é **medir** — antes de criar coluna nenhuma.

Passo 1 — testar a hipótese antes de treinar qualquer coisa

a piscina vale mais? duas linhas respondem

```
SELECT
  COUNTIF(REGEXP_CONTAINS(LOWER(titulo), r'piscina')) AS com_pisci
  ROUND(AVG(IF(REGEXP_CONTAINS(LOWER(titulo), r'piscina'), preco, NULL))) AS preco_me
  ROUND(AVG(IF(NOT REGEXP_CONTAINS(LOWER(titulo), r'piscina'), preco, NULL))) AS preco_me
FROM `SEU_PROJETO.anuncios.anuncios_gold`;
```

Aposte antes de rodar: imóvel com piscina no título é mais caro ou mais barato?

► **Resultado esperado** — não abra antes de rodar

Passo 2 — a coluna que a silver perdeu

Lembra do que morreu no caminho raw → silver? O **tipo do imóvel**. Um lote e um apartamento estão na mesma tabela, com o mesmo schema, e o modelo não tem como saber que são mercados diferentes. **O título sabe. Sempre soube.**

```

SELECT
  REGEXP_CONTAINS(LOWER(titulo), r'apartamento|apto|studio|kitnet|flat') AS eh_apartament
  REGEXP_CONTAINS(LOWER(titulo), r'casa|sobrado|mans[aão]o') AS eh_casa,
  REGEXP_CONTAINS(LOWER(titulo), r'lote|terreno|[aá]rea |fazenda|ch[aá]cara|s[ií]tio|rura
  COUNT(*) AS anuncios,
  ROUND(APPROX_QUANTILES(preco, 100)[OFFSET(50)]) AS mediana_preco,
  ROUND(APPROX_QUANTILES(preco / area_util, 100)[OFFSET(50)]) AS mediana_reais_por_m2
FROM `SEU_PROJETO.anuncios.anuncios_gold`
GROUP BY 1, 2, 3
ORDER BY anuncios DESC;

```

► **Resultado esperado — tipo × preço** — não abra antes de rodar

Passo 3 — a tabela gold_texto

Seis colunas novas: três de **tipo** (que coisa é) e três de **qualidade** (o que ela tem). **Nenhuma delas usa o preço** — guarde essa frase para o bloco 8.

```

CREATE OR REPLACE TABLE `SEU_PROJETO.anuncios.gold_texto` AS
SELECT
  *,
  -- QUALIDADE: a hipótese do Sávio
  REGEXP_CONTAINS(LOWER(titulo), r'piscina') AS tem_piscir
  REGEXP_CONTAINS(LOWER(titulo), r'alto padr[aão]o|luxo|requintad|exclusiv') AS eh_alto_pa
  REGEXP_CONTAINS(LOWER(titulo), r'mobiliad|armári|armari|planejad') AS tem_mobiliad

  -- TIPO: a coluna que a silver perdeu, reconstruída do texto
  REGEXP_CONTAINS(LOWER(titulo), r'lote|terreno|[aá]rea |fazenda|ch[aá]cara|s[ií]tio|rura
  REGEXP_CONTAINS(LOWER(titulo), r'apartamento|apto|studio|kitnet|flat') AS eh_apartame
  REGEXP_CONTAINS(LOWER(titulo), r'casa|sobrado|mans[aão]o') AS eh_casa
FROM `SEU_PROJETO.anuncios.anuncios_gold`;

```

Cobertura medida (887 linhas): piscina **31** · alto padrão **80** · mobília **51** · terreno **120** · apartamento **104** · casa **382**. Repare na assimetria — uma feature que existe em 3,5% das linhas dificilmente move a métrica geral. Segure isso também: é a chave do bloco 8.

Passo 4 — retreinar: mesmo algoritmo, seis colunas a mais

modelo_preco_v3 — a única diferença são as seis últimas

```
CREATE OR REPLACE MODEL `SEU_PROJETO0.anuncios.modelo_preco_v3`  
OPTIONS (  
  model_type           = 'LINEAR_REG',    -- IGUAL ao bloco 5  
  input_label_cols     = ['preco'],       -- IGUAL ao bloco 5  
  data_split_method    = 'AUTO_SPLIT',  
  enable_global_explain = TRUE  
) AS  
SELECT  
  preco,  
  area_util, quartos, banheiros, suites, garagem, condominio, iptu,  
  bairro, eh_comercial,  
  tem_piscina, eh_alto_padrao, tem_mobilia,      -- <<< as seis novas  
  eh_terreno, eh_apartamento, eh_casa  
FROM `SEU_PROJETO0.anuncios.gold_texto`;
```

► **Resultado esperado — o placar dos três modelos** — não abra antes de rodar

Passo 5 — e a piscina volta

quem pesou no v3

```
SELECT * FROM ML.GLOBAL_EXPLAIN(MODEL `SEU_PROJETO0.anuncios.modelo_preco_v3`)  
ORDER BY attribution DESC;
```

► **Resultado esperado — quem pesou no v3** — não abra antes de rodar

Tudo isso saiu de **uma coluna de texto** que já estava na tabela desde a primeira aula. Sem GPU, sem embedding, sem LLM: REGEXP_CONTAINS e uma pergunta de negócio.

Checkpoint 5. A gold_texto existe no meu dataset, o modelo_preco_v3 treinou e eu vi o R² subir de 0,298 para 0,471.

Métrica boa demais não se comemora, se investiga.

Script: `06_armadilha.sql`. No bloco 7 o texto do título levou o R^2 a 0,47. Funcionou tão bem que a extensão natural seria ir mais fundo no texto — tem mais coisa escrita ali. Este bloco mostra por que esse caminho, sem critério, produz um modelo que parece bom e não serve para nada.

Passo 1 — minerar o título de novo

Alguns anunciantes escrevem o valor no próprio título:

REGEXP_EXTRACT — a "feature nova"

```
SELECT
  titulo,
  ROUND(preco) AS preco,
  REGEXP_EXTRACT(titulo, r'R\$\s*([\d\.]+(?:,\d+)?\')) AS valor_no_titulo
FROM `SEU_PROJETO0.anuncios.anuncios_gold`
WHERE REGEXP_CONTAINS(titulo, r'R\$')
LIMIT 15;
```

Medido na Gold: **40 anúncios** têm "R\$" no título, **36 com valor parseável** — e, na conferência, **33 batem com o preço da coluna**. O script completo ainda ensina o parse do número sujo: o dataset mistura "8200000.00" (formato americano) e "12.500.000,00" (formato BR) — texto é traíçoeiro, e *isso também é NLP*.

Passo 2 — retreinar com a feature (modelo_preco_v2) e comparar

os dois lado a lado

```
SELECT 'v1_sem_titulo' AS modelo, mean_absolute_error, r2_score
FROM ML.EVALUATE(MODEL `SEU_PROJETO0.anuncios.modelo_preco`)
UNION ALL
SELECT 'v2_com_titulo', mean_absolute_error, r2_score
FROM ML.EVALUATE(MODEL `SEU_PROJETO0.anuncios.modelo_preco_v2`);
```

► **Resultado esperado** — não abra antes de rodar

Empatou no EVALUATE e o explain diz que a coluna não pesa nada.

Então ela é inofensiva — pode deixar no modelo?

Antes de responder: **de onde saiu esse número extraído do título?**

Decompondo o erro em dois grupos

A métrica agregada esconde o problema. Basta separar em dois grupos:

decompondo o erro

```
SELECT
  preco_no_titulo IS NOT NULL          AS titulo_entregava_o_preco,
  COUNT(*)                             AS anuncios,
  ROUND(AVG(ABS(predicted_preco - preco))) AS erro_medio_absoluto
FROM ML.PREDICT(
  MODEL `SEU_PROJETO0.anuncios.modelo_preco_v2`,
  (SELECT * FROM `SEU_PROJETO0.anuncios.gold_com_titulo`)
)
GROUP BY 1;
```

► **Resultado esperado** — não abra antes de rodar

O checklist — leve este da aula

1. Essa feature existiria no momento da predição?
2. Ela foi preenchida depois do que eu quero prever?
3. Meu alvo foi derivado de alguma coluna? Ela ficou FORA?

Treinar modelo no BigQuery é uma linha de SQL.

O trabalho de verdade é tudo que veio **antes** do CREATE MODEL: desconfiar da silver, achar a fazenda de meio Goiânia, e desconfiar do modelo que acertou demais.

Isso nenhuma ferramenta faz por você.

► **Desafios**

METADE DO CAMINHO

Até aqui o material construiu o pipeline e treinou os modelos.

Daqui em diante o assunto muda: o desenho de tudo junto, o resto do cardápio do BQML e as ideias para o Trabalho 1.

9 O pipeline inteiro, em uma tela

Agora que cada caixa foi construída por você, o desenho faz sentido.

Até aqui foram sete scripts. Vistos de longe, eles são **um** caminho só — e cada bifurcação dele foi uma **decisão sua**, não do algoritmo:



versão grande, com as setas, para projetar e para exportar em PNG

O placar

modelo	o que mudou	MAE	R²
v0	o dado como estava	R\$ 13.835.427	-0,350

v1	Gold limpa (887 linhas)	R\$ 2.581.138	+0,298
v3	+ 6 features do título	R\$ 2.469.775	+0,471
v4	árvore no lugar da reta	R\$ 1.823.401	+0,556

Três alavancas, nesta ordem de impacto:

- 1. **Limpar o dado** (v0 → v1) — o erro caiu **5,4×**
- 2. **Criar feature** (v1 → v3) — o R² foi de 0,298 para 0,471, sem coletar nada novo
- 3. **Trocar o algoritmo** (v3 → v4) — o erro caiu mais 26%, mudando **uma palavra**

Qual das três alavancas você teria puxado primeiro?

Quase todo mundo aposta na terceira, porque é a que "parece de ML". Foi a primeira que decidiu o jogo — e é a que ninguém quer fazer. Se você tivesse feito **só** a troca de algoritmo, a árvore treinaria no anúncio de R\$ 1,4 bilhão e iria mal do mesmo jeito.

Algoritmo bom não salva dado ruim.

Trocar de família de modelo é trocar uma string. O caro é ter chegado até aqui.

Script: `07_modelos_prontos.sql`. Até aqui as melhorias vieram do **dado**. Falta a terceira alavanca: o **algoritmo**.

Parte A — árvore no lugar da reta

Compare com o `CREATE MODEL` do bloco 7. Mesmo `SELECT`, mesma tabela, mesmas colunas. Mudou **uma palavra**:

`07_modelos_prontos.sql` — uma palavra de diferença

```
CREATE OR REPLACE MODEL `SEU_PROJETO.anuncios.modelo_preco_arvore`  
OPTIONS (  
  model_type           = 'BOOSTED_TREE_REGRESSOR', -- <<< era 'LINEAR_REG'  
  input_label_cols     = ['preco'],  
  data_split_method    = 'AUTO_SPLIT',  
  enable_global_explain = TRUE  
) AS  
SELECT  
  preco, bairro, area_util, quartos, banheiros, suites, garagem,  
  condominio, iptu, eh_comercial,  
  tem_piscina, eh_alto_padrao, tem_mobilia,  
  eh_terreno, eh_apartamento, eh_casa  
FROM `SEU_PROJETO.anuncios.gold_texto`;
```

Por que a árvore ganha aqui. A `LINEAR_REG` é uma fórmula só, válida para o dataset inteiro: cada +1 m² vale sempre a mesma coisa. A árvore encadeia regras (*SE área > 400 E bairro = Bueno ENTÃO...*), então cada +1 m² vale **uma coisa** no terreno e **outra** no apartamento. O nosso mercado não é uma reta.

► **Resultado esperado — o placar final dos quatro modelos** — não abra antes de rodar

A mesma pergunta, duas respostas diferentes

Rode o `ML.GLOBAL_EXPLAIN` nos **dois** modelos de preço e compare:

o linear e a árvore, lado a lado

```
-- o linear (v3)
SELECT feature, ROUND(attribution, 0) AS atribuicao
FROM ML.GLOBAL_EXPLAIN(MODEL `SEU_PROJETO.anuncios.modelo_preco_v3`)
ORDER BY attribution DESC;

-- a árvore (v4)
SELECT feature, ROUND(attribution, 0) AS atribuicao
FROM ML.GLOBAL_EXPLAIN(MODEL `SEU_PROJETO.anuncios.modelo_preco_arvore`)
ORDER BY attribution DESC;
```

► **Contraintuitivo — leia com calma** — não abra antes de rodar

Parte B — treinar sem resposta nenhuma

Tudo até aqui foi **aprendizado supervisionado**: existia uma coluna certa (`preco` , `eh_comercial`) e o modelo tentava acertá-la. Agora a pergunta muda de "quanto vale este imóvel?" para "**que tipos de imóvel existem nessa base, afinal?**"

KMEANS — repare no que NÃO tem aqui

```
CREATE OR REPLACE MODEL `SEU_PROJETO.anuncios.segmentos_imoveis`
OPTIONS (
  model_type           = 'KMEANS',
  num_clusters         = 4,
  standardize_features = TRUE
) AS
SELECT preco, area_util, quartos, banheiros, garagem
FROM `SEU_PROJETO.anuncios.gold_texto`;
```

- Não existe `input_label_cols` . Não existe resposta certa. O KMeans joga os 887 imóveis num espaço de 5 dimensões e procura 4 nuvens.
- `num_clusters = 4` — quantos grupos você quer. Não existe número "certo": 4 é um chute informado que dá para explicar a um humano.
- `standardize_features` é obrigatório aqui. Preço está na casa dos milhões e quartos na das unidades: sem padronizar, "distância entre dois imóveis" vira só "diferença de preço" e as outras quatro colunas somem.

o KMeans não dá nome aos grupos — dar nome é com você

```

SELECT
  CENTROID_ID                                AS segmento,
  COUNT(*)                                    AS imoveis,
  CAST(APPROX_QUANTILES(preco, 100)[OFFSET(50)] AS INT64) AS preco_mediano,
  CAST(APPROX_QUANTILES(area_util, 100)[OFFSET(50)] AS INT64) AS area_mediana,
  ROUND(AVG(quartos), 1)                     AS quartos_medio,
  ROUND(AVG(garagem), 1)                     AS vagas_media,
  ROUND(100 * AVG(CAST(eh_comercial AS INT64)), 1) AS pct_comercial
FROM ML.PREDICT(
  MODEL `SEU_PROJETO0.anuncios.segmentos_imoveis`,
  TABLE `SEU_PROJETO0.anuncios.gold_texto`)
GROUP BY segmento
ORDER BY preco_mediano;

```

Que nome você daria ao segmento com 0 quartos e 12 vagas de garagem?

► **Resultado esperado — os quatro grupos** — não abra antes de rodar

Checkpoint 6. Treinei a árvore, vi o placar dos quatro modelos, e consigo explicar por que o linear e a árvore discordam sobre qual coluna é a mais importante.

► **Desafios**

11 Ideias para o Trabalho 1 + checklist

Modelo com métrica perfeita será tratado como vazamento até prova em contrário.

O Trabalho 1 é **11/09** — daqui a 7 dias. Escolha uma ideia (ou traga a sua), desenvolva sobre o seu dado e apresente ao Sávio. Cada ideia vem com o risco que o professor vai olhar primeiro:

Ideia	Alvo / técnica	Risco a vigiar
Preço por bairro — para onde a imobiliária deve olhar?	regressão por segmento + <code>gold_relatorio_bairros</code> (28/08) como base	bairro com 3 anúncios não é estatística
Detector de anúncio suspeito — preço fora da curva	regressão + análise de resíduo (<code>ABS(erro)</code> alto = suspeito)	outlier de dado ≠ oportunidade de negócio
Classificador comercial × residencial melhorado	<code>BOOSTED_TREE_CLASSIFIER</code> , <code>auto_class_weights</code>	rótulo veio da regex do título — título proibido
Detector de duplicatas — o mesmo imóvel 4× (visto no bloco 5!)	<code>ML.DISTANCE</code> / similaridade, ou <code>GROUP BY</code> esperto	quando é duplicata e quando é reajuste de preço?
Segmentação de mercado aprofundada — o bloco 10 achou 4 grupos com 5 colunas. E com as features de texto dentro? E sem o preço?	<code>KMEANS</code> sobre a <code>gold_texto</code>	cluster não tem rótulo: interpretar e nomear é com você
Mineração de texto própria — que outras palavras do título explicam preço? ("vista", "nascente", "condomínio fechado")	<code>REGEXP_CONTAINS</code> + o teste do bloco 7 antes de treinar	hipótese razoável não é hipótese verdadeira: meça
Preço por m² como alvo — modelo mais estável?	<code>LINEAR_REG</code> com <code>preco_por_m2</code> como label	comparar com o modelo de <code>preco</code> na MESMA métrica

Antes de entregar, passe por aqui

1. Cada coluna que virou feature **existiria** no momento da predição?
2. Nenhuma feature foi preenchida **depois** do que eu quero prever?
3. Se o meu alvo foi derivado de uma coluna, essa coluna ficou **fora** das features?
4. Comparei meu modelo com "chutar a média / a classe majoritária"?
5. Rodei `ML.EVALUATE` e sei explicar o que cada número significa?
6. Meu `ML.PREDICT` funciona com uma linha **nova**, que não estava no treino?

Scripts da aula

Arquivo	Bloco
<code>00_avaliar_modelo_de_hoje.sql</code>	1 — o julgamento do modelo de 28/08
<code>01_silver_mente.sql</code>	2 — auditoria da silver
<code>02_gold_etl.sql</code>	3 — cortes justificados + <code>eh_comercial</code>
<code>03_regressao.sql</code>	5 — <code>LINEAR_REG</code> sobre a Gold + comparação
<code>04_classificacao.sql</code>	6 — <code>LOGISTIC_REG</code> , alvo derivado do título
<code>05_features_texto.sql</code>	7 — NLP legítima: seis flags do título
<code>06_armadilha.sql</code>	8 — vazamento de alvo
<code>07_modelos_prontos.sql</code>	10 — <code>BOOSTED_TREE</code> e <code>KMEANS</code>
<code>material/pipeline.html</code>	9 — o desenho de tudo, para projetar

Para onde isso vai

- **18/09 e 25/09** — o dado chegando em tempo real: Pub/Sub e Dataflow
- **02/10 e 09/10** — quando o modelo precisa sair do warehouse: Vertex AI e Cloud Run
- **13/11 e 27/11** — juntar tudo num pipeline orquestrado com n8n (o agente de 27/11 chama exatamente o `ML.PREDICT` do bloco 5)
- **11/09** — o Trabalho 1, apresentado ao Sávio
- **04/12** — o Trabalho Final é exatamente isso, de ponta a ponta

Guarde este arquivo. Ele funciona offline, os botões **copiar** continuam funcionando, e o seu progresso nos checkboxes fica salvo neste navegador.